

EQUIPO ATOMIC CODE

Segmentando el código, construyendo la lógica

EJERCICIO 2.2 - DERIVACIÓN Y MODELADO

Gramática Libre de Contexto de Dibujo

Caso práctico: el alfabeto del genoma $\Sigma = \{ a, c, g, t \}$

Guía de referencia para la defensa

Lenguaje y Compiladores - Trimestre 2026-I - UNEG

Victor Vargas	C.I: 30.697.219
Keibel Guilarte	C.I: 28.726.605
Oriana Márquez	C.I: 31.354.299
Jeanny Monagas	C.I: 30.857.471

1. ¿Qué pide el ejercicio?

El enunciado (punto 2.2.1) pide diseñar una **Gramática Libre de Contexto (GLC)** que modele una herramienta de dibujo sobre el alfabeto del genoma $\Sigma = \{a, c, g, t\}$, y presentar al menos **5 ejemplos de derivación** de sus producciones para casos concretos (un cuadrado, una figura con varias partes, etc.).

La idea central: una **gramática genera un lenguaje**. Cada cadena que la gramática produce (formada solo por símbolos de Σ) es una palabra del lenguaje. En este ejercicio, cada palabra es además una secuencia de órdenes que, al interpretarse, **dibuja una figura**. Así demostramos en la práctica la relación gramática-lenguaje.

2. Conceptos teóricos que debes dominar

- **Gramática Libre de Contexto (GLC):** una cuádrupla $G = (N, \Sigma, P, S)$. N son los no terminales (variables), Σ los terminales (el alfabeto), P las producciones (reglas) y S el símbolo inicial.
- **Terminal vs no terminal:** los terminales (a, c, g, t) son los símbolos finales que aparecen en la cadena. Los no terminales ($S, CUADRADO, L...$) son variables que se reemplazan hasta desaparecer.
- **Producción:** una regla de la forma $A ::= \text{cadena}$. Indica que el no terminal A se puede reemplazar por esa cadena (notación BNF).
- **Derivación por la izquierda:** proceso de generar una palabra reemplazando siempre el no terminal que está MÁS A LA IZQUIERDA, paso a paso, hasta que solo quedan terminales.
- **Tipo 2 en la Jerarquía de Chomsky:** las GLC son las gramáticas de Tipo 2. Son necesarias para estructuras anidadas (lo que una expresión regular no puede).

3. Datos específicos usados en el código

3.1 Alfabeto y semántica de la tortuga

El alfabeto es $\Sigma = \{a, c, g, t\}$. Cada terminal es una orden para una tortuga gráfica (estilo Logo / L-System):

```
a -> avanzar 1 unidad TRAZANDO (dibuja la línea)
c -> girar 90° a la derecha (clockwise)
g -> girar 90° a la izquierda (antihorario)
t -> avanzar 1 unidad SIN trazar (lápiz levantado)
```

3.2 La gramática completa (en notación BNF)

Esta es la gramática implementada en el diccionario GRAMATICA del código:

```
<S>          ::= <CUADRADO> | <RECTANGULO> | <ELE> | <ESCALERA> | <DOSCUADROS>
<CUADRADO>   ::= <L> c <L> c <L> c <L>
<RECTANGULO> ::= <L> c <M> c <L> c <M>
<ELE>        ::= c <M> g <L>
<ESCALERA>   ::= <P> <ESCALERA> | <P>
<DOSCUADROS> ::= <CUADRADO> t t t <CUADRADO>
<L>          ::= a | a <L> (lado de longitud >= 1)
```

<M>	::= a a <M>	(lado de longitud >= 1)
<P>	::= a c a g	(un peldaño de escalera)

- **L y M** son lados de longitud variable: por recursión (a <L>) generan tantos avances como se quiera. Aquí está el poder de la recursión de las GLC.
- **ESCALERA** es recursiva (<P> <ESCALERA>): genera tantos peldaños como haga falta.
- **DOSCUADROS** usa el terminal **t** (lápiz levantado) para separar dos figuras: es la prueba de que los 4 símbolos del alfabeto se utilizan.

3.3 Parámetros de las 5 figuras de ejemplo

a) CUADRADO (lado 3)	w = aaacaaacaaacaaa
b) RECTÁNGULO (5 x 3)	w = aaaaacaaacaaaaacaaa
c) FIGURA EN L (4 y 3)	w = caaagaaaa
d) ESCALERA (3 peldaños)	w = acagacagacag
e) DOS CUADRADOS (con salto, usa 't')	w = aacaacaacaatttaacaacaacaa

4. Explicación del algoritmo

4.1 Motor de derivación por la izquierda (función derivar)

El programa parte de la forma sentencial **['S']** y va aplicando una lista de pasos (la 'receta'). En cada paso:

- Busca el **no terminal más a la izquierda** de la forma actual.
- Verifica que la producción a aplicar **pertenezca realmente a la gramática** (rigor formal: si no existe, lanza un error).
- Reemplaza ese no terminal por el lado derecho de la producción.
- Guarda la nueva forma sentencial en el historial (para mostrar la derivación).

Cuando ya no quedan no terminales, la forma final es la **palabra w** del lenguaje. Las funciones `receta_cuadrado`, `receta_rectangulo`, etc. solo construyen la secuencia de producciones a aplicar para cada figura.

4.2 Intérprete de la tortuga (función dibujar)

La cadena w se recorre símbolo a símbolo manteniendo una **posición (x, y)** y una **dirección (dx, dy)** que empieza apuntando al Este (1, 0). La clave es cómo se implementan los giros de 90° como una rotación del vector dirección:

```
a: x += dx ; y += dy ; se marca el punto (trazo)
t: x += dx ; y += dy ; NO se marca (lápiz levantado)
c: (dx, dy) = (dy, -dx)      # giro 90° a la derecha
g: (dx, dy) = (-dy, dx)     # giro 90° a la izquierda
```

Como todos los giros son de 90° y los avances de 1 unidad, el trazo siempre es ortogonal y los puntos caen sobre una cuadrícula entera. La función `render` convierte ese conjunto de puntos en el dibujo ASCII, y `dibujar_turtle` lo traza en una ventana gráfica real con el módulo `turtle` de Python.

5. Recorrido del código (qué hace cada parte)

- **GRAMATICA:** diccionario que define las producciones (N, Σ, P, S).
- **derivar(receta):** motor de derivación por la izquierda; devuelve (cadena, historial).
- **formatear(forma):** muestra una forma sentencial con los no terminales entre $\langle \rangle$.
- **expandir_lado / receta_*:** construyen la lista de producciones de cada figura.
- **dibujar(cadena):** intérprete de la tortuga; devuelve el conjunto de puntos trazados.
- **render(puntos):** convierte los puntos en la cuadrícula ASCII.
- **dibujar_turtle / _trazar_turtle:** dibujan las figuras en la ventana gráfica.
- **ejecutar():** imprime, para cada figura, la derivación, la cadena y el dibujo.

6. Análisis de los resultados

6.1 Derivación completa de ejemplo: el cuadrado (lado 3)

Así genera la gramática el cuadrado, reemplazando paso a paso el no terminal más a la izquierda. Observa cómo $\langle L \rangle$ se expande por recursión hasta dar tres 'a':

```
<S>
=> <CUADRADO>
=> <L> c <L> c <L> c <L>
=> a <L> c <L> c <L> c <L>
=> a a <L> c <L> c <L> c <L>
=> a a a c <L> c <L> c <L>
=> a a a c a <L> c <L> c <L>
=> a a a c a a <L> c <L> c <L>
=> a a a c a a a c <L> c <L>
=> a a a c a a a c a <L> c <L>
=> a a a c a a a c a a <L> c <L>
=> a a a c a a a c a a a c <L>
=> a a a c a a a c a a a c a <L>
=> a a a c a a a c a a a c a a <L>
=> a a a c a a a c a a a c a a a
```

La palabra generada es **w = aaacaaacaaacaaa** (15 símbolos). Léida por la tortuga: avanza 3, gira derecha, y repite 4 veces, cerrando el cuadrado.

6.2 Las 5 figuras: cadena generada y dibujo

a) CUADRADO (lado 3)

```
w = aaacaaacaaacaaa
```

```
#####
##    ##
##    ##
#####
```

b) RECTÁNGULO (5 x 3)

w = aaaaacaacaaaaacaaa

```
#####
##      ##
##      ##
#####
```

c) FIGURA EN L (4 y 3)

w = caaagaaaa

```
##
##
##
#####
```

d) ESCALERA (3 peldaños)

w = acagacagacag

```
####
  ####
    ####
      ##
```

e) DOS CUADRADOS (con salto, usa 't')

w = aacaacaacaatttaacaacaaca

```
#####
##  ##
#####
```

```
#####
##  ##
#####
```

Interpretación: cada cadena de letras a/c/g/t no significa nada por sí sola, pero al pasarla por el intérprete produce una figura concreta. Esto evidencia que la gramática define un lenguaje cuyas palabras tienen un significado (un dibujo).

7. Cómo defenderlo (preguntas probables)

- **¿Por qué es libre de contexto y no regular?** Porque usa recursión para lados de longitud arbitraria y estructuras compuestas (DOSCUADROS); las producciones tienen un solo no terminal a la izquierda, que es la definición de Tipo 2.
- **¿Qué pasa si la longitud del lado cambia?** Solo se aplican más veces las producciones recursivas $\langle L \rangle ::= a \langle L \rangle$; la gramática es la misma.
- **¿Para qué sirve el terminal t?** Para levantar el lápiz y dibujar figuras separadas (DOSCUADROS); así se usan los 4 símbolos del alfabeto del genoma.
- **¿Qué es la derivación por la izquierda?** Reemplazar siempre el no terminal más a la izquierda;

garantiza un orden determinista y reproducible de la generación.

8. Cómo ejecutar el programa

```
# muestra derivaciones + dibujo ASCII y abre la ventana grafica (turtle)
python glc_genoma.py

# omite la ventana grafica (solo consola)
python glc_genoma.py --no-turtle
```